Exp-4

Implement document storage and top-k retrieval using a vector database.

Aim:

To build a functional text retrieval system that converts unstructured documents into dense numerical vector embeddings, indexes them inside a vector database using L2 (Euclidean) distance, and performs semantic top-k nearest neighbor search to find contextually relevant text without relying on exact keyword matching

Algorithm:

1. Initialization: Load a pre-trained embedding model (all-MiniLM-L6-v2) and define a raw corpus of documents. [1]
2. Vectorization (Embedding Generation):
    - Convert text documents into numerical vector representations of fixed dimensionality (d).
    - Convert the outputs into float32 Numpy arrays (required by FAISS). [1, 2, 3]
3. Indexing & Storage:
    - Initialize a FAISS flat index (IndexFlatL2) using the appropriate dimensions (d).
    - Store/insert the document vectors into the index database. [1, 2]
4. Query Execution:
    - Convert an unstructured incoming text query into a vector using the exact same embedding model.
    - Set the constraint parameter k for the maximum number of closest documents to retrieve.
5. Top-k Retrieval:
    - Traverse the index using L2 distance metric calculation to compute similarity.
    - Return an array of the top k minimum distances alongside their corresponding original document indices. [1]

Code:

```
import numpy as np
```

```
import faiss
from sentence_transformers import SentenceTransformer
documents = [
    "Vector databases store high-dimensional machine learning embeddings.",
    "Machine learning applications often rely on deep neural networks.",
    "The recipe for a perfect chocolate cake requires high-quality cocoa powder.",
    "Data science pipelines clean, transform, and index unstructured data.",
    "Baking an apple pie is an excellent weekend activity for family fun.",
    "Semantic search evaluates context instead of exact keyword matching."
]
print("--- Step 1: Loading Embedding Model ---")
model = SentenceTransformer('all-MiniLM-L6-v2')
dimension = 384
print("\n--- Step 2: Generating Vector Embeddings & Ingestion ---")
document_embeddings = model.encode(documents)
document_embeddings = np.array(document_embeddings).astype('float32')
index = faiss.IndexFlatL2(dimension)
index.add(document_embeddings)  # Store vectors in the index
print(f"Successfully stored {index.ntotal} documents in the vector database.")
print("\n--- Step 3: Performing Top-K Semantic Retrieval ---")
query = "How do AI models search for data contextually?"
k = 2
query_embedding = model.encode([query])
query_embedding = np.array(query_embedding).astype('float32')
distances, indices = index.search(query_embedding, k)
print(f"Query: '{query}'\n")
print(f"Top-{k} Most Relevant Results:")
for i in range(k):
    doc_idx = indices[0][i]
```

```
    score = distances[0][i]
    print(f"Rank {i+1}:")
    print(f"  • Document: '{documents[doc_idx]}'")
    print(f"  • Distance Score (L2): {score:.4f}")
```

Output:

--- Step 1: Loading Embedding Model ---

--- Step 2: Generating Vector Embeddings & Ingestion --- Successfully stored 6 documents in the vector database.

--- Step 3: Performing Top-K Semantic Retrieval --- Query: 'How do AI models search for data contextually?' Top-2 Most Relevant Results: Rank 1: • Document: 'Semantic search evaluates context instead of exact keyword matching.' • Distance Score (L2): 0.6384 Rank 2: • Document: 'Vector databases store high-dimensional machine learning embeddings.' • Distance Score (L2): 0.9023